

code_dont_fail_me_now

Shahjalal University of Science and Technology

1 Geometry

1.1	Geometry Primitives	1
1.2	Point 2D	1
1.3	Line Intersection	1
1.4	Line Equation	1
1.5	Segment Distance	1
1.6	Segment Intersection	1
1.7	Side Of	1
1.8	On Segment	1
1.9	Linear Transformation	1
1.10	Circle Intersection	1
1.11	Circle Polygon Intersection	2
1.12	Circle Line	2
1.13	Circle Tangents	2
1.14	Minimum Enclosing Circle	2
1.15	Inside Polygon	2
1.16	Polygon Area	2
1.17	Polygon Center	2
1.18	Polygon Cut	2
1.19	Polygon Union	2
1.20	Manhattan MST	2
1.21	Convex Hull	3
1.22	Hull Diameter	3
1.23	Point Inside Hull	3
1.24	Line Hull Intersection	3
1.25	Closest Pair	3

1 Geometry

1.1 Geometry Primitives

```
#define EPS 1e-12
#define pi 3.1415926535897931159979634\
68544185161590576171875L
bool feq(const long double& a, const
    long double& b) {
    long double del = fabs1(a - b);
    if (del <= EPS) return true;
    return del <= EPS * max(fabs1(a),
        fabs1(b));
}
```

1.2 Point 2D

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```
template <class T>
int sgn(T x) {
    return (x > 0) - (x < 0);
}

template <class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y); }
    bool operator==(P p) const { return tie(x, y) == tie(p.x, p.y); }
    P operator+(P p) const { return P(x + p.x, y + p.y); }
    P operator-(P p) const { return P(x - p.x, y - p.y); }
    P operator*(T d) const { return P(x * d, y * d); }
    P operator/(T d) const { return P(x / d, y / d); }
    T dot(P p) const { return x * p.x + y * p.y; }
    T cross(P p) const { return x * p.y - y * p.x; }
    T cross(P a, P b) const { return (a - *this).cross(b - *this); }
    T dist2() const { return x * x + y * y; }
};
```

```
double dist() const { return sqrt((double)dist2()); }
// angle to x-axis in interval [-pi, pi]
double angle() const { return atan2(y, x); }
P unit() const { return *this / dist(); } // makes dist()=1
P perp() const { return P(-y, x); } // rotates +90 degrees
P normal() const { return perp().unit(); }
// returns point rotated 'a' radians ccw around the origin
P rotate(double a) const {
    return P(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a));
}
P project(P p) const { return *this * ((double)dot(p) / dist2()); }
double scalar_project(P p) const { return (double)dot(p) / dist(); }
P reflect(P p) const { return project(p) * 2 - p; }
// Orientation test: >0 for left turn, <0 for right turn, 0 if collinear
T side(P p1, P p2) const { return (p1 - *this).cross(p2 - *this); }
friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << ", " << p.y << ")";
}
// Same direction, length r (returns *this if degenerate)
P truncate(double r) const { double k = dist(); return !sgn(k) ? *this : *this * (r / k); }
};
```

1.3 Line Intersection

Description:

Usage:
auto res = lineInter(s1,e1,s2,e2); if (res.first == 1) cout << "intersection point at " << res.second << endl;

```
template <class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```

1.4 Line Equation

```
array<T, 3> line_equation(const Point<T> &b, const Point<T> &c) {
    array<T, 3> res;
    res[0] = b.y - c.y, res[1] = -(b.x - c.x), res[2] = b.x*c.y - b.y*c.x;
    if (res[0] < 0) res[0] = -res[0], res[1] = -res[1], res[2] = -res[2];
    else if (res[0] == 0 and res[1] < 0) res[1] = -res[1], res[2] = -res[2]; // for double use feq(res[0], 0)
    return res;
}
```

1.5 Segment Distance

Description:

Usage:
Point<double> a, b(2,2), p(1,1); bool onSegment = segDist(a,b,p) < 1e-10;

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
    if (s == e) return (p - s).dist();
    auto d = (e - s).dist2(), t = min(d, max(0, (p - s).dot(e - s)));
}
```

```
return ((p - s) * d - (e - s) * t).dist() / d;
}
```

1.6 Segment Intersection

Description:

Usage:
vector<P> inter = segInter(s1,e1,s2,e2); if (sz(inter)==1) cout << "segments intersect at " << inter[0] << endl;

```
template <class P>
vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b), oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}
```

1.7 Side Of

Description:

Usage:
bool left = sideOf(p1,p2,q)==1;

```
template <class P>
int sideOf(P& s, P& e, P& p) {
    return sgn(s.cross(e, p));
}

template <class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
    auto a = (e - s).cross(p - s);
    double l = (e - s).dist() * eps;
    return (a > l) - (a < -l);
}
```

1.8 On Segment

Description: Returns true iff p lies on the line segment from s to e.

Usage:
\texttt{(segDist(s,e,p)<=epsilon)} instead when using Point<double>.

```
template <class P>
bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

1.9 Linear Transformation

Description: Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

```
typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1, const P& q0, const P& q1, const P& r) {
    P dp = p1 - p0, dq = q1 - q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r - p0).cross(num), (r - p0).dot(num)) / dp.dist2();
}
```

1.10 Circle Line Intersection

```
template <class P>
bool circleLineIntersection(P c, double r, P a, P b) {
    auto d1 = (c-a).dist(), d2 = (c-b).dist();
    if (max(d1, d2) < r-1e-9) return false;
    ld dd;
```

```

if ((c - a).dot(b - a) > 0 && (c - b).
    dot(a - b) > 0) {
    dd = abs((b - a).cross(c - a)) / (b
    - a).dist();
} else {
    dd = min(d1, d2);
}
return dd <= r+1e-9;
}

```

1.11 Circle Intersection

```

typedef Point<double> P;
bool circleInter(P a, P b, double r1,
    double r2, pair<P, P>* out) {
    if (a == b) {
        assert(r1 != r2);
        return false;
    }
    P vec = b - a;
    double d2 = vec.dist2(), sum = r1 + r2
    , dif = r1 - r2,
    p = (d2 + r1 * r1 - r2 * r2) /
    (d2 * 2), h2 = r1 * r1 - p * p * d2
    ;
    if (sum * sum < d2 || dif * dif > d2)
        return false;
    P mid = a + vec * p, per = vec.perp()
    * sqrt(fmax(0, h2) / d2);
    *out = {mid + per, mid - per};
    return true;
}

```

1.12 Circle Polygon Intersection

```

typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.
    dot(q))
double circlePoly(P c, double r, vector<
    P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p) / d.dist2(), b = (
        p.dist2() - r * r) / d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a - sqrt(det)), t
        = min(1., -a + sqrt(det));
        if (t < 0 || 1 <= s) return arg(p, q)
        * r2;
        P u = p + d * s, v = q + d * (t - 1)
        ;
        return arg(p, u) * r2 + u.cross(v) /
        2 + arg(v, q) * r2;
    };
    auto sum = 0.0;
    rep(i, 0, sz(ps)) sum += tri(ps[i] - c
    , ps[(i + 1) % sz(ps)] - c);
    return sum;
}

```

1.13 Circle Line

```

template <class P>
vector<P> circleLine(P c, double r, P a,
    P b) {
    P ab = b - a, p = a + ab * (c - a).dot
    (ab) / ab.dist2();
    double s = a.cross(b, c), h2 = r * r -
    s * s / ab.dist2();
    if (h2 < 0) return {};
    if (h2 == 0) return {p};
    P h = ab.unit() * sqrt(h2);
    return {p - h, p + h};
}

```

1.14 Circle Tangents

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0. $O(n)$

```

template <class P>

```

```

vector<pair<P, P>> tangents(P c1, double
    r1, P c2, double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(),
    h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<P, P>> out;
    for (double sign : {-1, 1}) {
        P v = (d * dr + d.perp() * sqrt(h2)
        * sign) / d2;
        out.push_back({c1 + v * r1, c2 + v *
        r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}

```

1.15 Minimum Enclosing Circle

Description: Computes the minimum circle that encloses a set of points. $O(n)$

```

pair<P, double> mec(vector<P> ps) {
    shuffle(all(ps), mt19937(time(0)));
    P o = ps[0];
    double r = 0, EPS = 1 + 1e-8;
    rep(i, 0, sz(ps)) if ((o - ps[i]).dist
    () > r * EPS) {
        o = ps[i], r = 0;
        rep(j, 0, i) if ((o - ps[j]).dist()
        > r * EPS) {
            o = (ps[i] + ps[j]) / 2;
            r = (o - ps[i]).dist();
            rep(k, 0, j) if ((o - ps[k]).dist
            () > r * EPS) {
                o = ccCenter(ps[i], ps[j], ps[k]
                );
                r = (o - ps[i]).dist();
            }
        }
    }
    return {o, r};
}

```

1.16 Inside Polygon

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow. $O(n)$

Usage:
vector<P> v = {P{4,4}, P{1,2}, P{2,1}}; bool
in = inPolygon(v, P{3, 3}, false);

```

// returns 1e9 if the point is on the
// polygon
int winding_number(const vector<P>& p,
    const P& z) { // O(n)
    if (is_point_on_polygon(p, z)) return
    1e9;
    int n = p.size(), ans = 0;
    for (int i = 0; i < n; ++i) {
        int j = (i + 1) % n;
        bool below = p[i].y < z.y;
        if (below != (p[j].y < z.y)) {
            auto orient = z.cross(p[j], p[i]);
            if (orient == 0) return 0;
            if (below == (orient > 0)) ans +=
            below ? 1 : -1;
        }
    }
    return ans;
}
// -1 if strictly inside, 0 if on the
// polygon, 1 if strictly outside
int is_point_in_polygon(const vector<P>& p,
    const P& z) { // O(n)
    int k = winding_number(p, z);
    return k == 1e9 ? 0 : k == 0 ? 1 : -1;
}

```

1.17 Polygon Area

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

```

template <class T>
T polygonArea2(vector<Point<T>>& v) {
    T a = v.back().cross(v[0]);
    rep(i, 0, sz(v) - 1) a += v[i].cross(v
    [i + 1]);
}

```

```

return a;
}

```

1.18 Polygon Center

Description: Returns the center of mass for a polygon. Time: $O(n)$

```

typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0);
    double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(
    v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].
        cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}

```

1.19 Polygon Cut

```

vector<P> polygonCut(const vector<P>& &
    poly, P s, P e) {
    vector<P> res;
    for (int i = 0; i < (int)poly.size();
    ++i) {
        P cur = poly[i], prev = i ? poly[i -
        1] : poly.back();
        auto a = s.cross(e, cur), b = s.
        cross(e, prev);
        if ((a < 0) != (b < 0)) res.
        push_back(cur + (prev - cur) * (a /
        (a - b)));
        if (a < 0) res.push_back(cur);
    }
    return res;
}

```

1.20 Polygon Union

```

double rat(P a, P b) { return sgn(b.x) ?
    a.x / b.x : a.y / b.y; }
double polyUnion(vector<vector<P>>& poly
    ) {
    double ret = 0;
    for (int i = 0; i < (int)poly.size();
    ++i) for (int v = 0; v < (int)poly[
    i].size(); ++v) {
        P A = poly[i][v], B = poly[i][(v +
        1) % (int)poly[i].size()];
        vector<pair<double, int>> segs =
        {{0, 0}, {1, 0}};
        for (int j = 0; j < (int)poly.size()
        ; ++j) if (i != j) {
            for (int u = 0; u < (int)poly[j].
            size(); ++u) {
                P C = poly[j][u], D = poly[j][(u
                + 1) % (int)poly[j].size()];
                int sc = sideOf(A, B, C), sd =
                sideOf(A, B, D);
                if (sc != sd) {
                    double sa = C.cross(D, A), sb
                    = C.cross(D, B);
                    if (min(sc, sd) < 0) segs.
                    emplace_back(sa / (sa - sb), sgn(sc
                    - sd));
                } else if (!sc && !sd && j < i
                && sgn((B - A).dot(D - C)) > 0) {
                    segs.emplace_back(rat(C - A, B
                    - A), 1);
                    segs.emplace_back(rat(D - A, B
                    - A), -1);
                }
            }
        }
    }
    sort(segs.begin(), segs.end());
    for (auto& s : segs) s.first = min(
    max(s.first, 0.0), 1.0);
    double sum = 0;
    int cnt = segs[0].second;
    for (int j = 1; j < (int)segs.size()
    ; ++j) {
        if (!cnt) sum += segs[j].first -
        segs[j - 1].first;
        cnt += segs[j].second;
    }
    ret += A.cross(B) * sum;
}

```

```
    return ret / 2;
}
```

1.21 Extreme Vertex

```
// id of the vertex having maximum dot
// product with z
// polygon must be convex
// top - upper right vertex
// for minimum dot product negate z and
// return -p[id].dot(z)
int extreme_vertex(const vector<P>& p,
    const P& z, const int top) { // O(
    log n)
    int n = p.size();
    if (n == 1) return 0;
    auto ans = p[0].dot(z);
    int id = 0;
    if (p[top].dot(z) > ans) ans = p[top].
        dot(z), id = top;
    int l = 1, r = top - 1;
    while (l < r) {
        int mid = (l + r) / 2;
        if (p[mid + 1].dot(z) >= p[mid].dot(
            z)) l = mid + 1;
        else r = mid;
    }
    if (p[l].dot(z) > ans) ans = p[l].dot(
        z), id = l;
    l = top + 1, r = n - 1;
    while (l < r) {
        int mid = (l + r) / 2;
        if (p[(mid + 1) % n].dot(z) >= p[mid
            ].dot(z)) l = mid + 1;
        else r = mid;
    }
    l %= n;
    if (p[l].dot(z) > ans) ans = p[l].dot(
        z), id = l;
    return id;
}
```

1.22 Manhattan MST

```
vector<array<int, 3>> manhattanMST(
    vector<P> ps) {
    vector<int> id(ps.size());
    iota(id.begin(), id.end(), 0);
    vector<array<int, 3>> edges;
    for (int k = 0; k < 4; ++k) {
        sort(id.begin(), id.end(),
            [&](int i, int j) { return (ps[
                i] - ps[j]).x < (ps[j] - ps[i]).y;
            });
        map<int, int> sweep;
        for (int i : id) {
            for (auto it = sweep.lower_bound(-
                ps[i].y); it != sweep.end();
                sweep.erase(it++)) {
                int j = it->second;
                P d = ps[i] - ps[j];
                if (d.y > d.x) break;
                edges.push_back({d.y + d.x, i, j
                });
            }
            sweep[-ps[i].y] = i;
        }
        for (P& p : ps)
            if (k & 1)
                p.x = -p.x;
            else
                swap(p.x, p.y);
    }
    return edges;
}
```

1.23 Convex Hull

Description: Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull. $O(n \log n)$

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
    int n = pts.size();
    if (n <= 1) return pts;
    sort(all(pts));
    vector<P> h(n + 1);
    int s = 0, t = 0;
```

```
    for (int it = 2; it--; s = --t,
        reverse(all(pts)))
        for (P p : pts) {
            while (t >= s + 2 && h[t - 2].
                cross(h[t - 1], p) <= 0) t--;
            h[t++] = p;
        }
    return {h.begin(), h.begin() + t - (t
        == 2 && h[0] == h[1])};
}
```

1.24 Hull Diameter

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points). $O(n)$

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S
        [0]}});
    rep(i, 0, j) for (;;) j = (j + 1) % n
        {
            res = max(res, {(S[i] - S[j]).dist2
                (), {S[i], S[j]}});
            if ((S[(j + 1) % n] - S[j]).cross(S[
                i + 1] - S[i]) >= 0) break;
        }
    return res.second;
}
```

1.25 Point Inside Hull

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included. $O(\log N)$

```
typedef Point<ll> P;
bool inHull(vector<P>& l, P p, bool
    strict = true) {
    int n = l.size(), a = 1, b = n - 1, r
        = !strict;
    if (n < 3) return r && onSegment(l[0],
        l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap
        (a, b);
    if (sideOf(l[0], l[a], p) >= r ||
        sideOf(l[0], l[b], p) <= -r) return
        false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a)
            = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

1.26 Line Hull Intersection

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. `lineHull(line, poly)` returns a pair describing the intersection of a line with the polygon:

- $(-1, -1)$ if no collision,
- $(i, -1)$ if touching the corner i ,
- (i, i) if along side $(i, i + 1)$,
- (i, j) if crossing sides $(i, i + 1)$ and $(j, j + 1)$.

In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line. $O(\log N)$

```
#define cmp(i, j) sgn(dir.perp().cross(
    poly[(i) % n] - poly[(j) % n]))
#define extr(i) cmp(i + 1, i) >= 0 &&
    cmp(i, i - 1 + n) < 0
template <class P>
int extrVertex(vector<P>& poly, P dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m
            + 1, m);
        (ls < ms || (ls == ms && ls == cmp(
            lo, m)) ? hi : lo) = m;
    }
```

```
    }
    return lo;
}
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<
    P>& poly) {
    int endA = extrVertex(poly, (a - b).
        perp());
    int endB = extrVertex(poly, (b - a).
        perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    rep(i, 0, 2) {
        int lo = endB, hi = endA, n = sz(
            poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 :
                n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi)
                = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return {res[0],
        -1};
    if (!cmpL(res[0]) && !cmpL(res[1]))
        switch ((res[0] - res[1] + sz(poly)
            + 1) % sz(poly)) {
            case 0:
                return {res[0], res[0]};
            case 2:
                return {res[1], res[1]};
        }
    return res;
}
```

1.27 Half Plane Intersection

```
typedef Point<double> P;
using Line = pair<P, P>;
bool bad(Line a, Line b, Line c) {
    auto res = lineInter(a.first, a.second
        , b.first, b.second);
    return res.first > 0 && sideOf(c.first
        , c.second, res.second, EPS) < 0;
}
vector<P> halfPlaneIntersection(vector<
    Line>& L) {
    auto ang = [] (Line l) { return (l.
        second - l.first).angle(); };
    sort(all(L), [&](Line a, Line b) {
        return ang(a) < ang(b); });
    vector<Line> lines;
    for (int i = 0; i < L.size(); i++) {
        if (i && abs(ang[L[i]] - ang[L[i - 1]]) <
            EPS) {
            if (sideOf(L[i].first, L[i].second,
                lines.back().first, EPS) < 0) {
                lines.pop_back(); lines.
                    push_back(L[i]);
            }
        } else lines.push_back(L[i]);
    }
    int n = lines.size(), qf = 0, qb = 0;
    vector<Line> q(n);
    for (int i = 0; i < n; i++) {
        while (qb - qf >= 2 && bad(q[qb - 1], q[
            qb - 2], lines[i])) qb--;
        while (qb - qf >= 2 && bad(q[qf], q[qf
            + 1], lines[i])) qf++;
        q[qb++] = lines[i];
    }
    while (qb - qf >= 3 && bad(q[qb - 1], q[qb
        - 2], q[qf])) qb--;
    while (qb - qf >= 3 && bad(q[qf], q[qf
        + 1], q[qb - 1])) qf++;
    if (qb - qf < 3) return {};
    vector<P> poly;
    for (int i = qf; i < qb; i++) {
        int nxt = (i + 1 == qb) ? qf : i + 1;
        poly.push_back(lineInter(q[i].first,
            q[i].second, q[nxt].first, q[nxt].
                second).second);
    }
    return poly;
}
```

1.28 Minkowski Sum Dif

```
using P = Point<ld>;
void reorder(vector<P>& poly) {
    int pos = 0;
    for (int i = 1; i < (int)poly.size(); ++i)
        if (poly[i] < poly[pos]) pos = i;
    rotate(poly.begin(), poly.begin() + pos, poly.end());
}

vector<P> minkowski(vector<P> a, vector<P> b) {
    reorder(a), reorder(b);
    int n = a.size(), m = b.size(), ai = 0, bi = 0;
    a.push_back(a[0]), a.push_back(a[1]);
    b.push_back(b[0]), b.push_back(b[1]);
    vector<P> c;
    while (ai < n || bi < m) {
        c.push_back(a[ai] + b[bi]);
        if (ai < n && bi < m) {
            auto ori = (a[ai + 1] - a[ai]).cross(b[bi + 1] - b[bi]);
            if (ori > 1e-9) ai++;
            else if (ori < -1e-9) bi++;
            else ai++, bi++;
        } else if (ai < n) ai++;
        else bi++;
    }
    return c;
}

vector<P> minkowski_dif(vector<P> a, vector<P> b) {
    for (auto& p : b) p.x = -p.x, p.y = -p.y;
    return minkowski(a, b);
}
```

1.29 Maximum Inscribed Circle

```
ld maxInscribedCircle(vector<P>& poly) {
    ld lo = 0, hi = 1e6, ans = 0, L = 1e9;
    int n = poly.size();
    while (hi - lo >= 1e-12) {
        ld mid = lo + (hi - lo) / 2.0;
        vector<Line> l = {{-L, -L}, {L, -L}}, {{L, -L}, {L, L}}, {{L, L}, {-L, L}}, {{-L, L}, {-L, -L}};
        for (int i = 0; i < n; ++i) {
            auto a = poly[i], b = poly[(i + 1) % n];
            auto c = (b - a).unit().perp();
            l.push_back({a + c * mid, b + c * mid});
        }
        auto cur = halfPlaneIntersection(l);
        if (!cur.empty()) ans = lo = mid;
        else hi = mid;
    }
    return ans;
}
```

1.30 Rotating Calipers

```
// Outer loop advances edge (i -> i+1),
// inner loop advances antipodal pointer j
for (int i = 0, j = 1; i < n; ++i) {
    while ((p[(i + 1) % n] - p[i]).cross(p[(j + 1) % n] - p[j]) > 0) {
        j = (j + 1) % n;
    }
    // Process antipodal pair: p[i] (or edge i -> i+1) and p[j]
}
```

1.31 Closest Pair

Description: Finds the closest pair of points. $O(n \log n)$

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
    assert(sz(v) > 1);
    set<P> S;
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
```

```
int j = 0;
for (P p : v) {
    P d{1 + (1ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo) ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
}
return ret.second;
}
```